

Quadrangle

四边形不等式优化

四边形不等式优化利用的是状态转移方程中的决策单调性。

基础知识

考虑最简单的情形，我们要解决如下一系列最优化问题。

这里假定成本函数 C 可以在 $O(1)$ 时间内计算。

▼ 约定

动态规划的状态转移方程经常可以写作一系列最优化问题的形式。以 (1) 式为例，这些问题含有参数 i, j ，问题的目标函数和可行域都可以依赖于 i, j 。每一个问题都是在给定参数 i, j 时，选取某个可行解 k 来最小化目标函数的取值。为表述方便，下文将参数为 (i, j) 的最优化问题简称为「问题」，该最优化问题的可行解 k 称为「决策」，目标函数在最优解处取得的值则称为「状态」。同时，记问题 (i, j) 对应的最小最优决策点为 $opt(i, j)$ 。

在一般的情形下，这些问题总时间复杂度为 $O(n^3)$ 。这是由于对于问题 (i, j) ，我们需要考虑所有可能的决策 k 。而在满足决策单调性时，可以有效缩小决策空间，优化总复杂度。

- **决策单调性**：对于任意 $i < i', j < j'$ ，必然成立 $opt(i, j) \leq opt(i', j')$ 。

► 附注

最常见的判断决策单调性的方法是通过四边形不等式 (quadrangle inequality)。

- **四边形不等式**：如果对于任意 $i < i', j < j'$ 均成立

则称函数 C 满足四边形不等式 (简记为「交叉小于包含」)。若等号永远成立，则称函数 C 满足 **四边形恒等式**。

如果没有特别说明，以下都会保证 C 满足四边形不等式。四边形不等式给出了一个决策单调性的充分不必要条件。

▼ 定理 1

若 C 满足四边形不等式，则问题 (1) 满足决策单调性。

► 证明

四边形不等式可以理解在合理的定义域内， C 的二阶混合差分 $C(i, j) + C(i', j') - C(i, j') - C(i', j)$ 非正。

利用决策单调性，有两种常见算法可以将算法复杂度优化到 $O(n^2)$ 。

分治

要求解所有状态，只需要求解所有最优决策点。为了对所有 (i, j) 求解，首先计算 $C(i, i)$ ，而后分别计算 $C(i, j)$ 和 $C(i', j')$ 上的，注意此时已知前半段的 opt 必然位于 i 和 $opt(i, j)$ 之间 (含端点)，而后半段的 opt 必然位于 $opt(i, j)$ 和 j 之间 (含端点)。对于两个子区间，也类似处理，直至计算出每个问题的最优决策。在分治的过程中记录搜索的上下边界，就可以保证算法复杂度控制在 $O(n^2)$ 。递归树层数为 $\log n$ ，而每层中，单个决策点至多计算两次，所以总的计算次数是 $O(n^2)$ 。

▼ 核心代码



C++Python

```
1 int w(int j, int i);
2
3 void DP(int l, int r, int k_l, int k_r) {
4     int mid = (l + r) / 2, k = k_l;
5     // 求状态f[mid]的最优决策点
6     for (int j = k_l; j <= min(k_r, mid - 1); ++j)
7         if (w(j, mid) < w(k, mid)) k = j;
8     f[mid] = w(k, mid);
9     // 根据决策单调性得出左右两部分的决策区间，递归处理
10    if (l < mid) DP(l, mid - 1, k_l, k);
11    if (r > mid) DP(mid + 1, r, k, k_r);
12 }

1 def DP(l, r, k_l, k_r):
2     mid = int((l + r) / 2)
3     k = k_l # 求状态f[mid]的最优决策点
4     for i in range(k_l, min(k_r, mid - 1)):
5         if w(i, mid) < w(k, mid):
6             k = i
7     f[mid] = w(k, mid) # 根据决策单调性得出左右两部分的决策区间，递归处理
8     if l < mid:
9         DP(l, mid - 1, k_l, k)
10    if r > mid:
11        DP(mid + 1, r, k, k_r)
```

二分队列

注意到对于每个决策点，能使其成为最小最优决策点的问题 必然构成一个区间。可以通过单调队列记录到目前为止每个决策点可以解决的问题的区间，这样，问题的最优解自然可以通过队列中记录的决策点计算得到。算法大致如下。

▼ 核心代码



C++

```

1 int val(int j, int i);
2 int lt[N], rt[N], f[N];
3 deque<int> dq;
4 // 初始化队列
5 dq.emplace_back(1);
6 lt[1] = 1;
7 rt[n] = n;
8 // 顺次考虑所有问题和决策
9 for (int j = 1; j <= n; ++j) {
10 // 出队
11 while (!dq.empty() && rt[dq.front()] < j) {
12     dq.pop_front();
13 }
14 // 计算
15 f[j] = val(dq.front(), j);
16 // 入队
17 while (!dq.empty() && val(j, lt[dq.back()]) < val(dq.back(), lt[dq.back()])) {
18     dq.pop_back();
19 }
20 if (dq.empty()) {
21     dq.emplace_back(j);
22     lt[j] = j + 1;
23     rt[j] = n;
24 } else if (val(j, rt[dq.back()]) < val(dq.back(), rt[dq.back()])) {
25     if (rt[dq.back()] < n) {
26         dq.emplace_back(j);
27         lt[j] = rt[dq.back()] + 1;
28         rt[j] = n;
29     }
30 } else {
31     int ll = lt[dq.back()];
32     int rr = rt[dq.back()];
33     int i;
34     // 二分
35     while (ll <= rr) {
36         int mm = (ll + rr) / 2;
37         if (val(j, mm) < val(dq.back(), mm)) {
38             i = mm;
39             rr = mm - 1;
40         } else {
41             ll = mm + 1;
42         }
43     }
44     rt[dq.back()] = i - 1;
45     dq.emplace_back(j);
46     lt[j] = i;
47     rt[j] = n;
48 }
49 }

```

掌握这一算法，需要理解如下要点：

- 队列需要记录到目前为止每个可行的决策点 和能够解决的问题区间左右端点 和 构成的 **三元组**。对于给定区间内的问题， 应该是到目前为止考虑过的决策点中最小最优的（以下简称最优决策）。每时每刻，队列中存储的决策未必是连续的，但是尚未解决的问题应该是队列中存储的问题区间的不交并。
- **初始化**：将首个决策放于队列中，并记录它对于所有问题都是最优的。
- 类似于单调队列，每次考虑下一个决策 的时候，都需要进行出队和入队操作。
- **出队**：当所有决策 都考虑结束后，问题 的解就是队列中首个满足 的决策点 。此时可以弹出所有满足 的队首。由于决策单调性，弹出的决策也不会是后续问题的最优决策。
- **入队**：要对决策 进行入队时，首先比较它和队尾的决策 。

- 如果对于问题 P ，将入队的决策 d 比已有的决策 d' 更优，即 $d < d'$ 时，则弹出队尾的决策 d' 。此操作持续到队尾的决策 d'' 比 d 对于问题 P 更优时为止。
- 如果队列已空，入队 d ，即认为决策 d 是尚未解决的所有问题的最优解。
- 如果队尾决策 d'' 对于问题 P 同样优于将入队的决策 d ，那么当 $d < d''$ 时，入队 d ，表示 d 是对于问题 P 的最优解，否则，不需要入队，因为它并不比已有的决策更优。
- 最后的情形是，队尾决策 d'' 比起要入队的决策 d 对于问题 P 更优，而对于问题 P' 更劣，那么，需要通过二分找到最小的 x 使得 $d < d''$ ，将队尾的区间右端点修改为 x ，并入队 d 。

类似于单调队列，每个决策点至多入队一次，出队一次。这里，出队是 $O(1)$ 的，而入队是 $O(\log n)$ 的（可能需要二分），所以总的时间复杂度是 $O(n \log n)$ 。

▼ 例题 1: 「POI2011」Lightning Conductor

给定一个长度为 n 的序列 $a_1, a_2, \dots, a_n$ ，要求对于每一个 i ，找到最小的非负整数 b_i 满足

- 思路
- 实现

区间分拆问题

考虑将某个区间拆分成若干个子区间的问题。形式化地说，将给定区间 $[l, r]$ 拆分成 k 个子区间，其中 $l = l_1 < l_2 < \dots < l_k = r$ ，以及 $l_i < l_{i+1}$ 对任意 i 都成立。对于给定拆分，成本为 $\sum_{i=1}^{k-1} c(l_i, l_{i+1})$ 。问题要求最小化这一成本。可以列出如下的 1D1D 状态转移方程。

这里， $c(l, r)$ 注意到，只要 $c(l, r)$ 满足四边形不等式，必然满足四边形不等式，因为第一项并不包括 $c(l, r)$ 和 $c(l, r)$ 的交叉项，在混合差分时会消去。但是由于成本函数依赖于前面的子问题，这一转移只能顺序计算，所以通常只适合应用二分队列算法。算法复杂度为 $O(n^2)$ 。

限制区间个数的情形

上述问题可以加强为限制区间个数的情形，即问题指定将区间拆分成 k 个子区间。此时需要将拆分后的区间个数作为转移状态的一维。相应地，有 2D1D 状态转移方程如下。

这里， $c(l, r)$ 对任意 $l < r$ 都成立。和上文同样的道理，这里的 $c(l, r)$ 必然满足四边形不等式。此时对于第 k 层的计算，并不再依赖于该层的结果，所以对于每一层，都可以通过分治或者二分队列的方法进行计算，此时算法复杂度为 $O(n^2 \log k)$ 。

对于这一问题，利用决策单调性，实际上还存在其他的优化算法。第二种优化思路依赖于如下结果。这种优化算法和下文详细描述 Knuth 优化算法十分相似。

▼ 定理 2

若 $c(l, r)$ 满足四边形不等式，则对于问题 (2) 成立。

► 证明

利用这一结果，我们可以限制决策 d 的搜索范围。算法实现时，对 d 正向遍历，对 d' 逆向遍历，在之前已确定的上下界范围内暴力搜索 d 就可以保证 d 的算法复杂度。

► 注意

最后一种优化方法来源于如下的观察。

▼ 定理 3

若 $\mathcal{S}$ 满足四边形不等式，则问题 (2) 的最优解 f_n 是关于 n 的凸函数。

► 证明

这一结论保证了可以通过 wqs 二分（国外称 Alien's trick）的方法解决此问题。具体来说，考虑带参的成本函数，解决不限制区间个数的问题，求得其最优解为 f_n 。随着实数 λ 递增，相应的最优区间的数目单调递减，故而可以通过二分的方法找到恰使得最优区间个数等于 k 的参数 λ ，则原问题最优解为 f_n 。这里的实数 λ 可以看作区间个数限制的 Lagrange 乘子。该算法的实现有很多细节，可以参考 [专门介绍 wqs 二分的文章](#)。这一算法的时间复杂度为 $O(n \log n)$ ，这里 C 为某一常数。

对于限制区间个数的区间分拆问题的三种算法，在不同的数据范围时表现各有优劣，需要结合具体的题目选择合适的算法。

▼ 例题 2: [P4767 \[IOI2000\] 邮局 加强版](#) [P6246 \[IOI2000\] 邮局 加强版 加强版](#)

高速公路旁边有一些村庄。高速公路表示为整数轴，每个村庄的位置用单个整数坐标标识。没有两个在同样地方的村庄。两个位置之间的距离是其整数坐标差的绝对值。

邮局将建在一些，但不一定是所有的村庄中。为了建立邮局，应选择他们建造的位置，使每个村庄与其最近的邮局之间的距离总和最小。

你要编写一个程序，已知村庄的位置和邮局的数量，计算每个村庄和最近的邮局之间所有距离的最小可能的总和。

► 思路

► 实现 1，前文第二种优化，复杂度 $O(n^2)$

► 实现 2，wqs 二分，复杂度 $O(n \log n)$

区间合并问题

另一类可以通过四边形不等式优化的动态规划问题是区间合并问题，即要将 n 个长度为一的区间两两合并起来，直到得到区间 $[1, n]$ 。每次合并 $[l, r]$ 和 $[r+1, s]$ 时都需要支付成本 $cost(l, r, s)$ 。问题要求找到成本最低的合并方式。对于此类问题，有如下 2D1D 状态转移方程。

这里给定任意初始成本 $cost$ 。暴力算法的总复杂度为 $O(n^3)$ ，而当存在决策单调性时，可以优化至 $O(n^2)$ 的算法复杂度。这一算法最早由 Knuth 在解决最优二叉搜索树问题时提出，并由姚储枫进一步研究总结，在国外称为 Knuth's optimization 或 Knuth-Yao speedup。

除了四边形不等式以外，区间合并问题的决策单调性还要求成本函数满足区间包含单调性。

- **区间包含单调性：**如果对于任意 $l_1 < l_2 < r_1 < r_2$ 均成立

则称函数 $cost$ 对于区间包含关系具有单调性。

这实质是成本函数的一阶条件，即 $cost$ 关于 l 递减，关于 r 递增。

▼ 引理 1

若 $\mathcal{S}$ 满足区间包含单调性和四边形不等式，则状态 $f_{l, r}$ 满足四边形不等式。

► 证明

▼ 定理 4

若 $\mathcal{S}$ 满足区间包含单调性和四边形不等式，则问题 (3) 中最小最优决策 $f_{l, r}$ 满足

► 证明

利用这一结论，同样可以限制决策点 的搜索范围。在这里，正序遍历区间长度，再遍历具有同样长度的所有区间，暴力搜索 和 之间的所有 求得最优解 并记录最小最优决策。对于同样长度的所有区间，此算法中决策空间总长度是的，而可能的区间长度的数目同样是 的，故而总的算法复杂度为 的。

▼ 核心代码



C++Python

```
1 for (int len = 2; len <= n; ++len) // 枚举区间长度
2   for (int j = 1, i = len; i <= n; ++j, ++i) {
3     // 枚举长度为len的所有区间
4     f[j][i] = INF;
5     for (int k = opt[j][i - 1]; k <= opt[j + 1][i]; ++k)
6       if (f[j][i] > f[j][k] + f[k + 1][i] + w(j, i)) {
7         f[j][i] = f[j][k] + f[k + 1][i] + w(j, i); // 更新状态值
8         opt[j][i] = k; // 更新（最小）最优决策点
9       }
10  }
```

```
1 for len in range(2, n + 1): # 枚举区间长度
2   for i in range(len, n + 1):
3     # 枚举长度为len的所有区间
4     j = i - len + 1
5     f[j][i] = INF
6     for k in range(opt[j][i - 1], opt[j + 1][i] + 1):
7       if f[j][i] > f[j][k] + f[k + 1][i] + w(j, i):
8         f[j][i] = f[j][k] + f[k + 1][i] + w(j, i) # 更新状态值
9         opt[j][i] = k # 更新（最小）最优决策点
```

满足四边形不等式的函数类

为了更方便地证明一个函数满足四边形不等式，我们有以下几条性质：

性质 1：若函数 和 均满足四边形不等式（或区间包含单调性），则对于任意，函数 也满足四边形不等式（或区间包含单调性）。

性质 2：若存在函数 和 使得，则函数 满足四边形恒等式。当函数 和 单调增加时，函数 还满足区间包含单调性。

性质 3：设 是一个单调增加的凸函数，若函数 满足四边形不等式并且对区间包含关系具有单调性，则复合函数 也满足四边形不等式和区间包含单调性。

性质 4：设 是一个凸函数，若函数 满足四边形恒等式并且对区间包含关系具有单调性，则复合函数 也满足四边形不等式。

首先需要澄清一点，凸函数（Convex Function）的定义在国内教材中有分歧，此处的凸函数指的是下凸函数，即（可微时）一阶导数单调增加的函数。

► 证明

习题

- [Codeforces - Ciel and Gondolas](#)(Be careful with I/O!)
- [SPOJ - LARMY](#)
- [Codechef - CHEFAOR](#)
- [Hackerrank - Guardians of the Lunatics](#)

- [ACM ICPC World Finals 2017 - Money](#)

参考资料

- [noiau 的 CSDN 博客](#)
 - [Quora Answer by Michael Levin](#)
 - [Video Tutorial by "Sothe" the Algorithm Wolf](#)
 - [Divide and Conquer DP](#)
 - [Knuth's Optimization](#)
 - [Quadrangle Inequality Properties](#)
 - [王钦石《浅析一类二分方法》](#)
-

本页面最近更新：2024/9/24 14:27:05, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者：[StudyingFather](#), [c-forrest](#), [lr1d](#), [Marcythm](#), [NFLSCode](#), [billchenchina](#), [CCXXXI](#), [changwenxuan](#), [Chrogeek](#), [Elitedj](#), [Enter-tainer](#), [fps5283](#), [greyqz](#), [Henry-ZHR](#), [hsfzLZH1](#), [iamtwz](#), [izlyforever](#), [ksyx](#), [Menci](#), [MingqiHuang](#), [nanmenyangde](#), [opsiff](#), [ouuan](#), [ranwen](#), [shawlleyw](#), [sshwy](#), [TOMWT-qwq](#), [TrisolarisHD](#), [Xeonacid](#), [Yanjun-Zhao](#), [zryi2003](#), [zyf0726](#)

本页面的全部内容 [在 CC BY-SA 4.0 和 SATA 协议之条款下](#) 提供，附加条款亦可能应用